

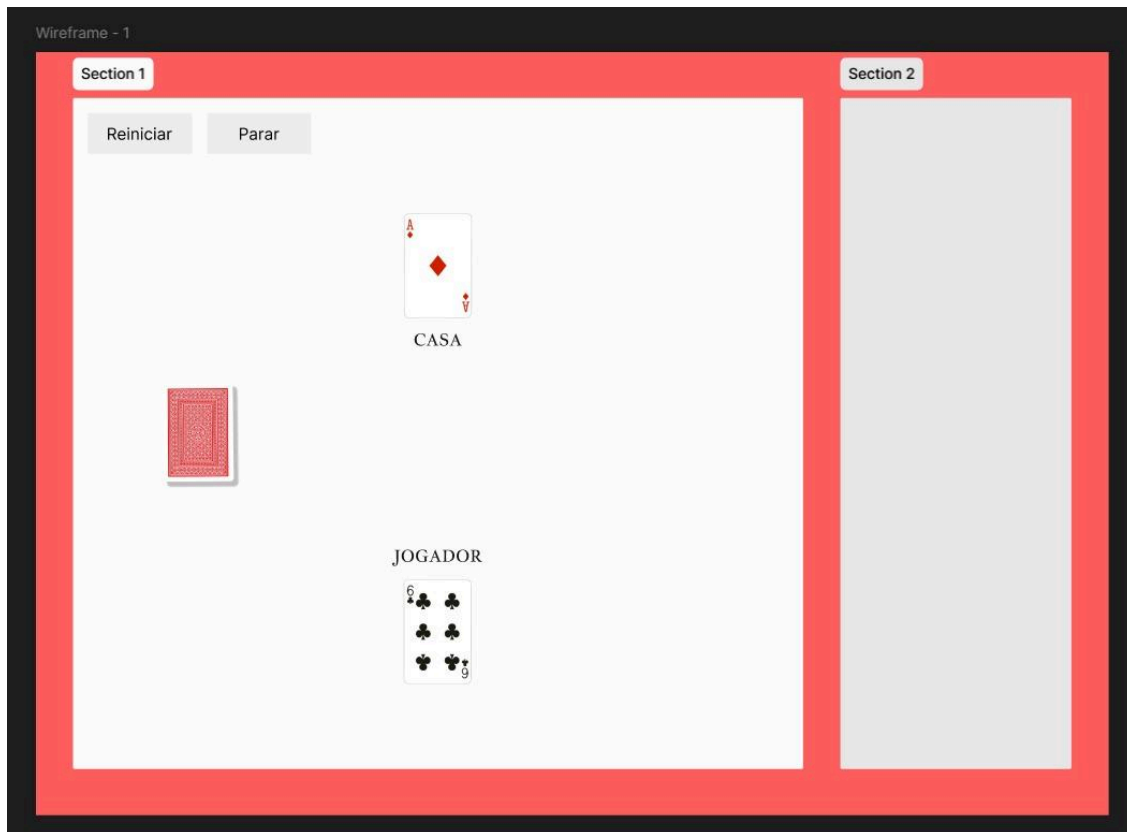
Daniel Pauleti 10723162
Lucas Yan 10437822
Nicholas Lopes 10723200
Victor Gargalhone 10723637

Projeto Pichu

Ideação

O website criado pelo grupo servirá para o usuário jogar Blackjack, ou 21, contra o programa desenvolvido no backend do projeto. O Javascript será usado para permitir o usuário a tomar ações no jogo, iniciar uma nova partida, e para guardar as variáveis e manter o funcionamento do jogo em si.

A imagem abaixo é um esboço de como o website deve aparentar.



O jogo será jogado contra a Casa, um adversário programático. O objetivo é obter a pontuação mais próxima de 21 sem ultrapassar este valor. O usuário e a Casa compram cartas de um mesmo baralho, tentando marcar mais pontos que o adversário. Porém, a compra de cartas não é obrigatória, e para isso existe a opção de “Parar”.

Exemplo: O usuário tirou as cartas 4, 9 e 5 totalizando 18. Ele não quer correr o risco de passar do limite de 21, então ele decide “Parar”. A Casa tira 6, 7 e 2 totalizando 15, e

precisa tirar um valor entre 19 e 21 para ganhar o jogo. Ela compra um 8, extrapolando o limite de 21 pontos, e perde o jogo.

As cartas Rei, Rainha e Valete valem 10 pontos na contagem. Outra regra especial é a combinação de uma dessas três cartas com um Ás, formando um Blackjack, que é a melhor mão possível, garantindo a vitória.

O usuário tem seu turno primeiro, comprando cartas até estourar ou parar. Em sequência, é a vez da Casa, que é obrigada a comprar cartas até ganhar do usuário ou perder.

Protótipo

O protótipo inicial contém as três partes principais do website: a área de jogo visual, a caixa de texto com o registro das ações tomadas no jogo, e os botões necessários para a interação por parte do usuário. Nenhuma função se encontra presente, e os aspectos visuais do website estão em sua forma mais básica.

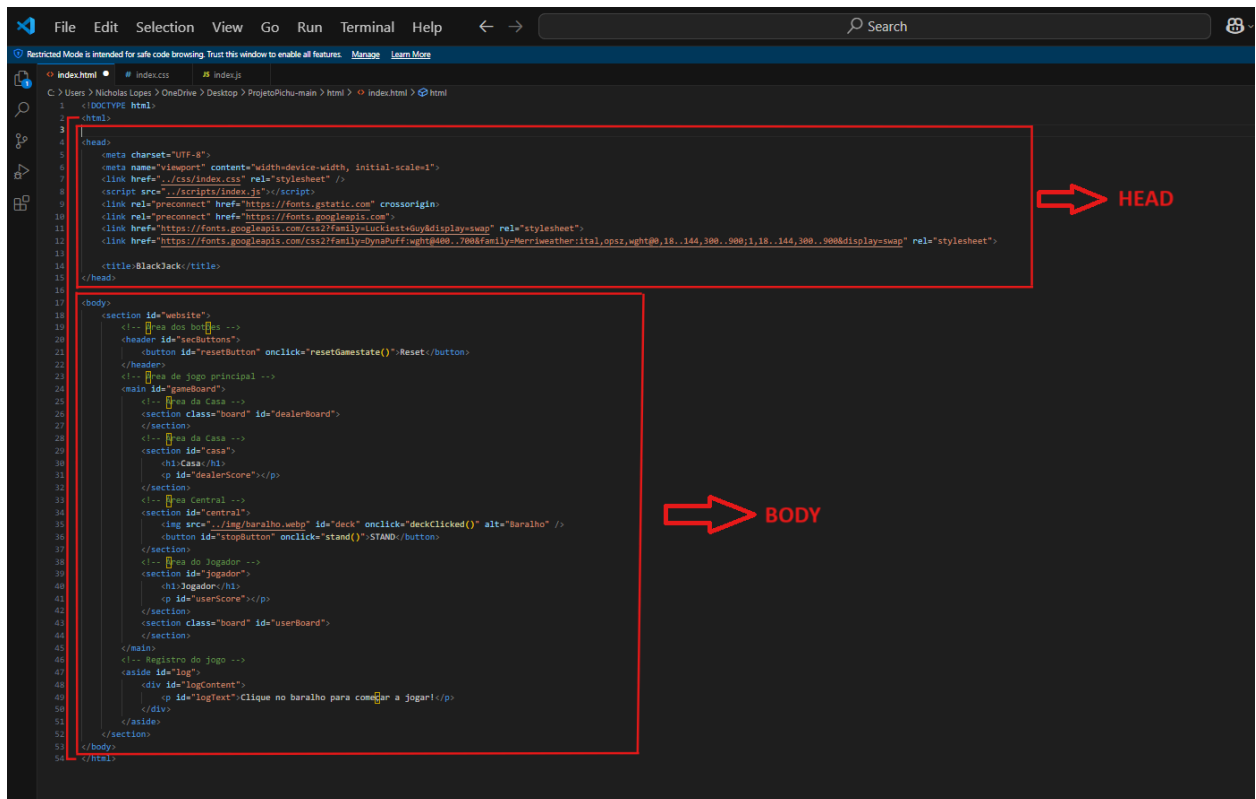
A Área de jogo incluirá os elementos visuais do jogo: as cartas em jogo e as pontuações da casa e do usuário. Será realizada através de imagens posicionadas dentro da área e elementos de texto que mudam de comportamento dependendo do estado do jogo.

O Registro será um elemento de texto com a função scroll, que também acompanhará as ações tomadas no jogo. Em contraste com a área de jogo, o foco será no detalhe e compreensão, permitindo que o usuário leia e acompanhe as pontuações e jogadas prévias.

Os botões serão o foco das interações do usuário com o site, permitindo que este tome ações no jogo e inicie novas partidas.

Tutorial

O código a seguir cria a estrutura de uma página web de um protótipo de um jogo de cartas. Vou explicar cada parte de forma bem simples:



```
1 <!DOCTYPE html>
2 <html>
3
4 <head>
5   <meta charset="UTF-8">
6   <meta name="viewport" content="width=device-width, initial-scale=1">
7   <link href="css/index.css" rel="stylesheet" />
8   <script src="scripts/index.js"></script>
9   <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
10  <link rel="preconnect" href="https://fonts.gstatic.com" />
11  <link href="https://fonts.googleapis.com/css2?family=Inter:wght@400;700&family=Merriweather:ital,opsz,wght@18..144,300..900;18..144,300..900&display=swap" rel="stylesheet">
12  <link href="https://fonts.googleapis.com/css2?family=DynaPuff:wght@400;700&family=Merriweather:ital,opsz,wght@18..144,300..900;18..144,300..900&display=swap" rel="stylesheet">
13
14  <title>BlackJack</title>
15 </head>
16
17 <body>
18   <section id="website">
19     <!-- Área dos botões -->
20     <header id="buttons">
21       <button id="resetButton" onclick="resetGameState()">Reset</button>
22     </header>
23     <!-- Área de jogo principal -->
24     <main id="gameBoard">
25       <!-- Área da Casa -->
26       <section class="board" id="dealerBoard">
27         <!-- Área da Casa -->
28         <section id="casa">
29           <h1>Casa</h1>
30         </section>
31         <p id="dealerScore"></p>
32       </section>
33       <!-- Área Central -->
34       <section id="central">
35         
36         <button id="stopButton" onclick="stand()">STAND</button>
37       </section>
38       <!-- Área do Jogador -->
39       <section id="jogador">
40         <h1>Jogador</h1>
41         <p id="userScore"></p>
42       </section>
43       <section class="board" id="userBoard">
44       </section>
45     </main>
46     <!-- Registro de jogo -->
47     <aside id="log">
48       <div id="logContent">
49         <p id="logText">Clique no baralho para começar a jogar!</p>
50       </div>
51     </aside>
52   </section>
53 </body>
54 </html>
```

Usando a linguagem de programação HTML foi criado a estrutura base que contém a tag `<head>` (Configurações e metadados (não visível)) e a tag `<body>` (Conteúdo visível e interativo da página). Dentro da tag head foi incluído as tags metas que configuram a página; as tags link que conectam o documento html em documentos como CSS e Javascript que estilizam a página e criam interações entre a página e o usuário e também a tag title (título) que dará um nome para a página mostrando esse nome na barra da guia da web.

```
15 <body>
16 >   <section id="website">...
43   </section>
44 </body>
```

```
16 <section id="website">
17   <!-- Área dos botões -->
18   <header id="buttons">
19     <button id="resetButton">Reset</button>
20   </header>
```

Dentro do body (corpo da página, ou seja, tudo aquilo que ficará visível na página da web) foi criado uma tag section para separar dentro do corpo um “espaço” que engloba todo o conteúdo da página. A Área dos Botões contém um botão com id="resetButton", que servirá para reiniciar o jogo.

```
22     <main id="gameBoard">
23         <!-- Área da Casa -->
24         <section class="board" id="dealerBoard">
25         </section>
26         <h1>Área da Casa</h1>
27         <!-- Área Central -->
28         <section id="central">
29             
30         </section>
31         <!-- Área do Jogador -->
32         <h1>Área do Jogador</h1>
33         <section class="board" id="userBoard">
34         </section>
35     </main>
```

A **Área Principal do Jogo** foi implementada na tag main com id="gameBoard" e é dividida em três partes:

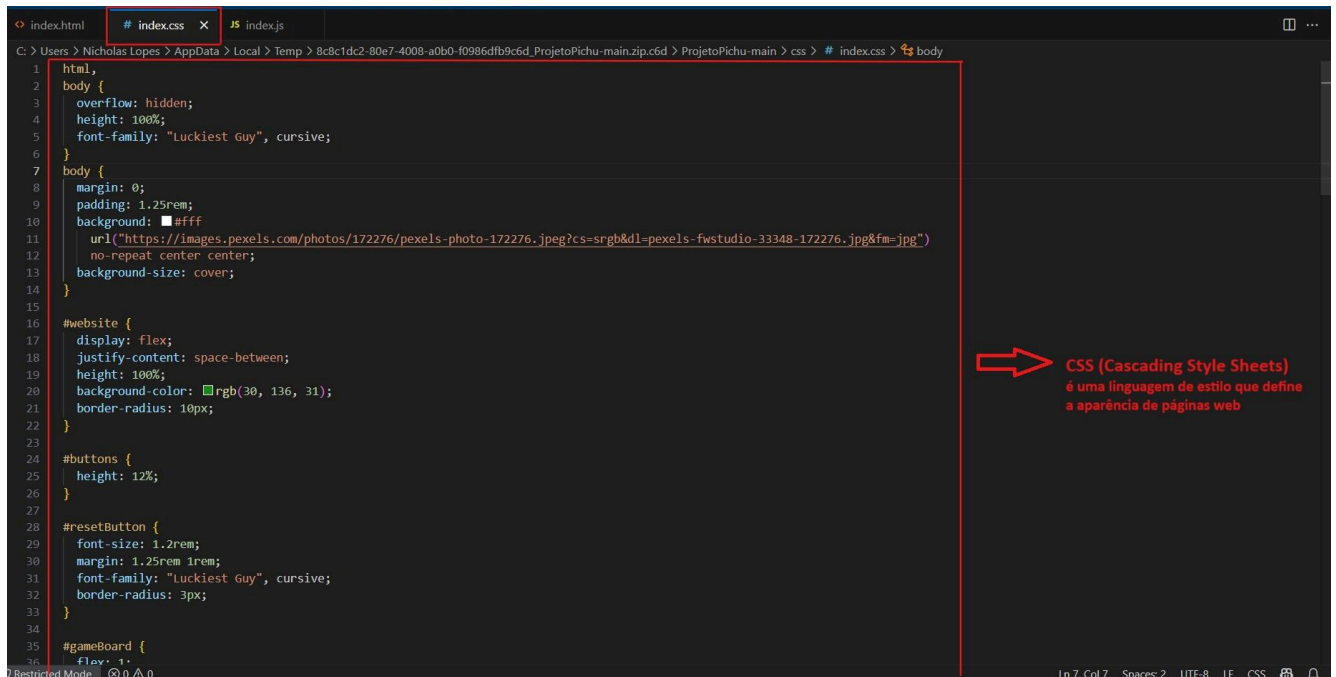
1. Área da Casa (<h1>Área da Casa</h1> + <section class="board">)
 - Representa o local onde as cartas da "casa" (banco/cassino) serão exibidas.
2. Área Central (<section id="central">)
 - Contém uma imagem (), que representa o baralho do jogo.
3. Área do Jogador (<h1>Área do Jogador</h1> + <section class="board">)
 - Onde as cartas do jogador serão exibidas.

```
37     <aside id="log">
38         <h1>Registro do Jogo</h1>
39         <div id="logContent">
40             <p id="logText">Clique no baralho para começar a jogar!</p>
41         </div>
42     </aside>
```

E por fim temos o “**Registro do Jogo**” (<aside id="log">)

- Um painel de registros para mostrar mensagens do jogo.
- Contém parágrafos <p> com textos informativos, indicando que ele pode ser usado para armazenar logs das ações durante o jogo.

2-1. Estilização geral



Com o uso de CSS conectado a um documento html através de um link dentro da tag head como foi explicado no início do tutorial, é possível transformar uma página HTML simples em algo bonito e organizado. O CSS funciona aplicando **estilos** aos elementos HTML de uma página. Ele segue um sistema baseado em **seletores** e **propriedades** para definir como os elementos devem aparecer.

```
1 body,
2 h1,
3 p {
4   margin: 0;
5 }
```

Reset global do margin para evitarmos espaços indesejáveis.

```
7 html,
8 body {
9   height: 102%;
10 }
11 body {
12   padding: 1.25rem;
13   background: #fff
14   url("https://images.pexels.com/photos/172276/pexels-photo-172276.jpeg?cs=srgb&dl=pexels-fwstudio-33348-172276.jpg&fm=jpg")
15   no-repeat center center;
16 }
17 #resetButton,
18 #stopButton,
19 #gameBoard {
20   font-family: "Luckiest Guy", cursive, sans-serif;
21 }
```

height: 100% garante que o html e o body ocupem toda a altura da tela (em meio ao desenvolvimento adicionamos mais 2% para ajustar o layout do gameBoard). Font-family especifica

a fonte de um elemento para a "Luckiest Guy" como a principal, e se ela não estiver disponível, usa uma fonte do tipo sans-serif.

Padding: 1.25rem; adiciona um pequeno espaço interno ao redor do conteúdo da página e é nesse espaço que adicionamos uma background image pela Url(...) e, caso a imagem não carregue, o background com o valor "#fff" definirá uma cor de fundo branca. No-repeat impede que a imagem se repita; center center centraliza a imagem na tela e background-size: cover; faz com que a imagem cubra toda a tela, mantendo a proporção.

```
#website {
  display: flex;
  justify-content: space-between;
  height: 100%;
  background-color: #2e8b57;
  border-radius: 10px;
}

#buttons {
  height: 12%;
}

#resetButton {
  font-size: 1.2rem;
  margin: 1.25rem 1rem;
  font-family: "Luckiest Guy", cursive;
  border-radius: 3px;
}
```

```
#gameBoard {
  flex: 1;
  display: flex;
  flex-direction: column;
  justify-content: space-around;
  font-size: 1.25rem;
  padding: 3rem;
}

#gameBoard h1 {
  text-align: center;
  color: white;
  font-weight: 400;
  font-style: normal;
  margin: 0;
}
```

Aqui foi criados quatro id (identificador) que são identificados pelo carácter “#” para adicionar estilos em css a um único elemento do html. Esse trecho do código estiliza um site com um container principal (#website) de fundo verde e bordas arredondadas. Dentro dele, há um botão de reinício (#resetButton) estilizado e um quadro de jogo (#gameBoard) organizado em colunas, com um título (h1) branco e centralizado. O layout usa Flexbox para distribuir os elementos de forma responsiva.

```

42     #stopButton {
43         border-radius: 20px;
44         border: 5px solid rgb(255, 255, 255);
45         background-color: #ff0000;
46         color: rgb(255, 255, 255);
47         font-size: 2.2rem;
48         height: 6.5rem;
49         width: 9.5rem;
50         padding: 0.5rem;
51     }

```

Estilização do botão “STAND”, que ficará ao lado do baralho. Ele tem um background-color: #ff0000 (vermelho) uma borda branca e arredondada.

```

.board {
    border: 0.5rem double black;
    border-radius: 25px;
    height: 20%;
    text-align: center;
    background-color: white;
}

.card {
    height: 7.5rem;
}

#central {
    display: flex;
    justify-content: space-around;
    text-align: center;
    width: 100%;
    height: 20%;
}

#deck {
    height: 100%;
}

#log {
    overflow: auto;
    border: 0.5rem double black;
    border-radius: 25px;
    font-size: 1rem;
    width: 20%;
    height: 92%;
    background-color: #f7f7f7;
    margin: 1rem;
}

```

Esse trecho contém duas classes identificadas pelo carácter “.”; seguidas pelo seletor: “board” e “card” e dentro do parte chaves está o estilo do CSS a ser aplicado em todos os elementos do HTML que estão atrelados a essas classes. Esse trecho de código CSS está configurando um layout onde há elementos com bordas duplas e arredondadas, áreas flexíveis para centralização, e um log que possui rolagem automática quando o conteúdo excede o espaço disponível. A aparência é limpa, com bordas e fundo bem definidos, e o uso de flexbox para organizar os elementos de forma responsiva.

2-1. Media Queries

Utilizamos Media Queries para adaptar o layout do website de acordo com o tamanho da tela, onde dependendo da largura usada pelo usuário podemos facilitar a visualização e a usabilidade em diferentes dispositivos, como por exemplo, um celular ou um tablet.

```
82 > @media (max-width: 768px) { ...  
127 }
```

Max-width é usado para mudar o layout para dispositivos com largura menor de 768 pixels, todo código que mudará de disposição dentro deste intervalo está dentro das chaves.

```
82 @media (max-width: 768px) {  
83   body {  
84     overflow: auto;  
85     box-sizing: border-box;  
86   }  
87  
88   #website {  
89     height: auto;  
90     display: flex;  
91     flex-direction: column;  
92   }
```

No body, anteriormente nós não queríamos o scroll, mas agora no responsivo vamos precisar dele e ele é implementado pelo “overflow: auto”. Já a propriedade “box-sizing: border-box” utilizamos para que o padding de 1.25rem colocado lá no início, esteja dentro do valor definido da “height” e não crie um espaço vazio indesejado. Veja a diferença:

- com valor padrão (box-sizing: content-box;)



- com `box-sizing: border-box;`



Utilizamos “display flex” e “flex-direction: column” para dispor os elementos dentro da section principal em coluna e mudamos a height para auto a fim de permitir que o site tenha uma altura maior que visível, possibilitando o scroll.

```
94  #buttons {  
95    height: 8%;  
96  }  
97  #resetButton {  
98    font-size: 1.1rem;  
99    padding: 5px 8px;  
100 }
```

Diminuímos o tamanho do botão e de sua fonte, e, adicionamos um padding para facilitar a utilização pelo usuário.

```

103 #gameBoard {
104     padding: 2rem;
105     gap: 1.25rem;
106 }
107 .board {
108     height: 7.5rem;
109     margin: 0.5rem;
110 }
111 #deck {
112     height: 7.5rem;
113 }

```

Ajustamos a altura do “board” e do “deck” e ajustamos o espaçamento do gameBoard.

```

115 #log {
116     height: 15rem;
117     width: 90%;
118     background-color: rgba(200, 232, 200, 0.6);
119 }
120 }

```

Por fim ajustamos o “log” onde será registrado o andamento do jogo, definimos um height para ele não ocupar muito espaço do layout e um width para ele ocupar o espaço do #website. Mudamos a cor de fundo dele para destacar a Área da casa e a Área do jogador.

3. Lógica do Jogo com JavaScript

Agora, implementamos a lógica do jogo em JavaScript. O código definirá as funções essenciais para o funcionamento do jogo de 21, como comprar uma carta, calcular a pontuação e verificar o vencedor.

```

1  // Let's go gambling!!!
2  // BlackJack with Jack Black
3  // Chicken Jockey
4
5  //CONSTANTS
6  //Create deck array
7  const deckBase = ['AC', '2C', '3C', '4C', '5C', '6C', '7C', '8C', '9C', '0C', 'JC', 'QC', 'KC',
8    'AD', '2D', '3D', '4D', '5D', '6D', '7D', '8D', '9D', '0D', 'JD', 'QD', 'KD',
9    'AH', '2H', '3H', '4H', '5H', '6H', '7H', '8H', '9H', '0H', 'JH', 'QH', 'KH',
10   'AS', '2S', '3S', '4S', '5S', '6S', '7S', '8S', '9S', '0S', 'JS', 'QS', 'KS'];
11
12 //Operate on clone
13 let deck = deckBase.slice();
14
15 //Create hands
16 let handUser = [];
17 let handDealer = [];
18
19 //Game started and ended flag
20 let gameStarted = false;
21 let gameEnded = false;
22 |
23 //Aces value flag
24 let acesLow = false;
25
26 //Score variables
27 let scoreUser = 0;
28 let scoreDealer = 0;
29

```

Essas são as variáveis centrais:

- deckBase: baralho original com 52 cartas (A = Ás, C = Copas, D = Ouros, H = Corações, S = Espadas, e "0" representa o 10).
- deck: cópia do baralho usada no jogo atual.
- handUser e handDealer: mãos dos jogadores.
- gameStarted / gameEnded: flags para controle de estado do jogo.
- acesLow: controla se o Ás vale 1 (ao invés de 11).
- scoreUser / scoreDealer: pontuação atual.

```

30 //Create score array (LUCAS) - Dicionário de valores
31 cardsScoreBase = {}
32   "A": 11, "2": 2, "3": 3, "4": 4, "5": 5, "6": 6, "7": 7, "8": 8, "9": 9, "0": 10, "J": 10, "Q": 10, "K": 10
33 }
34
35 //Set score array to base array
36 cardsScore = {...cardsScoreBase}; //Create a copy of the base array to operate on

```

Define o valor de cada carta. O Ás vale 11 inicialmente, mas pode ser alterado para 1 se necessário (para evitar que um jogador estoure).

```

55 function randomCard() {
56   //console.log("Dealing random card")
57
58   const randomIndex = Math.floor(Math.random() * deck.length); //Random index
59   const card = deck[randomIndex]; //Random card
60   deck.splice(randomIndex, 1); //Remove card from deck
61   //console.log(card);
62   //console.log(deck);
63   return card;
64 }

```

A função "randomCard()" pega uma carta aleatória do baralho e a remove do deck:

```

66 //Deal to hand (UNFINISHED)
67 function dealRandomCard(player, hidden = false, logDraw = true) {
68     //console.log("Dealing random card to " + player);
69
70     const card = randomCard();
71     //Function will not work unless turnActive is set
72     //turnActive = "user";
73
74     if (player === "user") { //Check if player is user or dealer
75         handUser.push(card);
76         displayCard(card, "user");
77         //log(messageDictionary("hit", player));
78         if (logDraw) { //Log dealing card message
79             log(cardMessage(card)); //Log dealing card message
80         }
81     }
82     else if (player === "dealer") {
83         handDealer.push(card);
84         displayCard(card, "dealer", hidden);
85         if (logDraw) { //Log dealing card message
86             log(cardMessage(card));
87         }
88     }
89     else {
90         console.log("Invalid player");
91     }
92 }

```

A função “dealRandomCard” (player, hidden = false, logDraw = true)

Distribui uma carta para o user ou dealer. Pode ser oculta (hidden) e opcionalmente registrar o evento no log.

“resetCardDisplay()” e “resetScoreDisplay()” Limpa as cartas e pontuação da interface, enquanto “log(message)” e “resetLog()” mostram mensagens no registro de ações (“log”) que aparece no canto direito da tela do jogo

```

93 //Show current gamestate (debug)
94 function showGameState() {
95     console.log("User hand: " + handUser);
96     console.log("Dealer hand: " + handDealer);
97 }
98 //Retrieve card value from array
99 function getCardValue(card) {
100     return cardsScore[card[0]];
101 }
102
103 //Calculate score (LUCAS) - Calcula valor total dado uma mão e o dicionário de valores
104 function calculateScore(player = "") {
105     let hand = player === "user" ? handUser : handDealer; //Get hand based on player
106     let score = 0;
107     hand.forEach(card => score += getCardValue(card)); //Sum values of cards in hand
108     updateScore(player, score); //Update score variable
109     if (!(turnActive == "user" & player == "dealer")) { //Prevent score message for dealer on game start
110         log(messageDictionary("score", player) + scoreMessage(score)); //Log score message
111     }
112     if (bust(score)) { //Check if score is bust
113         if (aces(hand) & !acesLow) { //Check if hand has aces and if ace value has not been altered
114             switchAceValue(); //Switch ace value to 1
115             return calculateScore(player); //Recalculate score
116         } else {
117             log(messageDictionary("bust", player)); //Log bust message
118             endGame(player === "user" ? false : true); //End game, checking who busted
119         }
120     }
121     if (blackjack(score)) {
122         if (!(turnActive != "dealer" & player == "dealer")) { //Prevent dealer blackjack on game start
123             log(messageDictionary("blackjack", player)); //Log blackjack message
124             endGame(player === "user" ? true : false); //End game, checking who has blackjack
125         }
126     }
127     return score;
128 }
129

```

A função “calculateScore(player)” soma o valor total das cartas da mão de um jogador:

- Usa “getCardValue(card)” para pegar o valor de cada carta.
- Verifica Blackjack (21) ou Bust (passou de 21).
- Se tiver um Ás e o jogador estourar, troca o valor do Ás de 11 para 1 com switchAceValue() e recalcula.

“switchAceValue()” Troca o valor do Ás para 1 se necessário

“startGame()” Inicia o jogo: Dá 2 cartas para cada jogador (uma do dealer fica oculta). Exibe as cartas e define a vez do usuário.

“endGame(userWin)” Finaliza o jogo com base em quem venceu.

“houseAction()” Controla a jogada da casa (dealer). Se a pontuação do dealer for menor que ou igual à do jogador, ele continua pegando cartas até ganhar ou estourar.

Funções de Reset resetDeck(), resetHands(), resetAces() e resetGameState() — recomeçam o jogo do zero.

Aqui começa a parte do código que controla o que será mostrado na tela, podemos chamar de “Funções Frontend” ou seja, é a parte visual de uma aplicação, que permite que os usuários interajam.

```
275 //FUNCTIONS - Frontend
276 //Display card
277 function displayCard(card, player, hidden = false) {
278     let imageSection;
279     if (player == "user") { //Check if player is user or dealer
280         imageSection = document.getElementById("userBoard");
281     } else if (player == "dealer") {
282         imageSection = document.getElementById("dealerBoard");
283     }
284
285     const cardImage = document.createElement("img");
286
287     if (hidden) {
288         cardImage.src = "../img/verso.jpg"; //Hidden card image
289         cardImage.setAttribute("data-card", card); //Store card name
290         cardImage.classList.add("hidden-card"); //Add class for selection later
291     } else {
292         cardImage.src = `../img/${card}.jpg`; //Get card image
293     }
294
295     cardImage.classList.add("card"); //Add class to card
296     imageSection.appendChild(cardImage); //Add card to image section
297 }
298
299 //Reveal hidden card
300 function revealHiddenCards() {
301     document.querySelectorAll(".hidden-card").forEach((img) => {
302         const cardName = img.getAttribute("data-card"); //Get stored card name
303         img.src = `../img/${cardName}.jpg`; //Update src to show the actual card
304         img.classList.remove("hidden-card"); //Remove hidden class
305     });
306 }
307
```

displayCard
(card, player,
hidden)
Mostra a
imagem da
carta na tela.

“revealHiddenCards()” Revela as cartas escondidas do dealer.

```

328 //Card to message - Function to convert card name to message
329 function cardMessage(card) {
330     let cardValue = card.slice(0, -1); //Get the value of the card
331     let cardSuit = card.slice(-1); //Get the suit of the card
332
333     var suits = { //Get suit name
334         'C': 'Paus',
335         'D': 'Ouros',
336         'H': 'Copas',
337         'S': 'Espadas'
338     };
339
340     var values = { //Get value name
341         '2': "2",
342         '3': "3",
343         '4': "4",
344         '5': "5",
345         '6': "6",
346         '7': "7",
347         '8': "8",
348         '9': "9",
349         '0': "10",
350         'J': "Valete",
351         'Q': "Rainha",
352         'K': "Rei",
353         'A': "As"
354     };
355
356     return `${values[cardValue]} de ${suits[cardSuit]}`;
357 }
358
359 //Score to message - Function to convert score to message
360 function scoreMessage(score) {
361     return `${score} pontos`;
362 }

```

```

364 //Message dictionary - Function called to return messages based on game events
365 function messageDictionary(trigger, player = "") {
366     var messageBase = { //Implement better dictionary
367         "start": "O jogo começou!", //Add messages to every relevant function
368         "hit": "pediu mais uma carta!",
369         "stand": "parou!",
370         "bust": "estourou!",
371         "blackjack": "fez um blackjack!",
372         "win": "ganhou!",
373         "lose": "perdeu!",
374         "reset": "Preparando um novo jogo...",
375         "score": "tem um total de ",
376     };
377
378     var playerName = {
379         "user": "Você",
380         "dealer": "A casa",
381         "" : ""
382     }
383
384     return `${playerName[player]}${messageBase[trigger]}`
385 }
386
387 //Change score display
388 function displayScore(player, hiddenCards = false) {
389     let score = calculateScore(player); //Calculate score
390     let handId = player === "user" ? "userScore" : "dealerScore"; //Get hand id based on player
391
392     if (hiddenCards) { //If hidden cards are present, set score to "?"
393         score = "?";
394     }
395
396     document.getElementById(handId).textContent = score;
397 }
398

```

Essas duas funções (cardMessage(card) e messageDictionary(trigger, player)) são usadas para mostrar mensagens na caixa de registro de ações ("log") que aparece no canto direito da tela do jogo, mensagens que servem para atualizar o jogador sobre o andamento do jogo.

“displayScore(player, hiddenCards = false)” mostra a pontuação do jogador na interface (pode esconder no caso do dealer). E por fim a função “deckClicked()” chamada ao clicar no baralho: se o jogo já começou e for a vez do jogador, ele compra uma nova carta; se o jogo ainda não começou, inicia o jogo.